

The Ultimate Graphs Reference Guide

BFS to Bridges, Flows to Centroid Decomposition

Sahil Raj (After Dark)

Contents

1	Graph Fundamentals	5
1.1	Terminology	5
1.2	Graph Representations	5
1.2.1	Adjacency List (default choice)	5
1.2.2	Adjacency Matrix	5
1.2.3	Edge List	6
1.3	BFS — Breadth-First Search	6
1.4	DFS — Depth-First Search	6
1.5	Connected Components & Flood Fill	7
1.6	Cycle Detection	7
1.6.1	Undirected Graph (DFS)	7
1.6.2	Directed Graph (DFS + recursion stack)	7
1.7	Topological Sort	8
1.7.1	DFS-based (Postorder)	8
1.7.2	Kahn's Algorithm (BFS-based, detects cycles)	8
1.8	Bipartite Check	8
2	Shortest Paths	11
2.1	Algorithm Selector	11
2.2	Dijkstra's Algorithm	11
2.2.1	Dijkstra with Path Reconstruction	12
2.3	0-1 BFS	12
2.4	Bellman-Ford	13
2.4.1	SPFA — Shortest Path Faster Algorithm	13
2.5	Floyd-Warshall — All-Pairs Shortest Path	14
2.6	Multi-Source BFS	14
2.7	Shortest Path in a DAG (Topological Order DP)	15
3	Minimum Spanning Tree	17
3.1	MST Fundamentals	17
3.2	DSU — Disjoint Set Union (Union-Find)	17
3.3	Kruskal's Algorithm	18
3.4	Prim's Algorithm	18
3.5	Second Minimum Spanning Tree	19
3.6	Minimum Spanning Forest	19
3.7	Borůvka's Algorithm (Non-Standard)	20
4	SCCs, Bridges & Articulation Points	21
4.1	Strongly Connected Components (SCCs)	21
4.2	Kosaraju's Algorithm	21
4.3	Tarjan's Algorithm for SCCs	22
4.4	Bridges	22
4.5	Articulation Points (Cut Vertices)	23

4.6	Bridge Tree (Block-Cut Tree)	24
4.7	Eulerian Path and Circuit	24
5	Network Flow	25
5.1	Flow Network Basics	25
5.2	Ford-Fulkerson / Edmonds-Karp	25
5.3	Min-Cut from Max-Flow	26
5.4	Bipartite Matching	27
5.4.1	Simple DFS Matching (Augmenting Paths)	27
5.4.2	Hopcroft-Karp ($\mathcal{O}(E\sqrt{V})$)	27
5.5	Min-Cost Max-Flow (MCMF)	28
5.6	König's Theorem	29
6	Advanced Graph Algorithms	31
6.1	LCA — Lowest Common Ancestor	31
6.1.1	Binary Lifting	31
6.1.2	LCA + Path Max-Edge Queries	32
6.2	Euler Tour (DFS In/Out Times)	32
6.3	Heavy-Light Decomposition (HLD)	33
6.4	Centroid Decomposition	34
6.5	DSU on Tree (Small to Large / Dsu on Tree)	35
6.6	Rerooting DP	36
6.7	Virtual Tree (Auxiliary Tree)	37
6.8	Functional Graph	37
6.9	Algorithm Selector Cheat Sheet	38
7	Competitive Programming Graph Algorithms	39
7.1	2-SAT (Two-Satisfiability)	39
7.2	SCC Condensation & DAG DP	40
7.3	Dijkstra on State-Space Graphs	41
7.4	K-th Shortest Path (Yen's / Modified Dijkstra)	42
7.5	Hamiltonian Path (Bitmask DP)	42
7.6	Tree DP Patterns	43
7.6.1	Tree Diameter	43
7.6.2	Max Independent Set on Tree	43
7.6.3	Counting Paths with Length Exactly K (Centroid DP)	44
7.7	Graph Colouring	44
7.8	Bipartite Matching Applications	45
7.8.1	Minimum Path Cover in DAG	45
7.8.2	Maximum Independent Set (Bipartite)	45
7.8.3	Minimum Vertex Cover (Bipartite)	45
7.9	Common Gotchas in Graph CP	45

Chapter 1

Graph Fundamentals

1.1 Terminology

Pattern / Core Idea

A **graph** $G = (V, E)$: V = vertices, E = edges.

Directed (digraph) Edges have direction: $(u \rightarrow v) \neq (v \rightarrow u)$

Undirected $(u, v) = (v, u)$

Weighted Each edge has a cost $w(u, v)$

DAG Directed Acyclic Graph — no directed cycles

Simple No self-loops, no multi-edges

Connected Path exists between every pair of vertices

Degree $\deg(u)$ = edges incident to u

In/Out-degree Digraph: $\deg^-(u)$ / $\deg^+(u)$

1.2 Graph Representations

1.2.1 Adjacency List (default choice)

```
1 int n, m; // vertices, edges
2 vector<vector<int>> adj(n); // unweighted
3 vector<vector<pair<int,int>>> wadj(n); // weighted: {neighbour, weight}
4
5 // Build
6 cin >> u >> v >> w;
7 adj[u].push_back(v);
8 adj[v].push_back(u); // omit for directed
9 wadj[u].push_back({v, w});
```

Complexity

Space: $\mathcal{O}(V + E)$. Iterate neighbours: $\mathcal{O}(\deg(u))$. Use for sparse graphs.

1.2.2 Adjacency Matrix

```
1 int mat[505][505]; // mat[u][v] = weight (0 = no edge, INF = no edge for SSSP)
2 mat[u][v] = w;
3 mat[v][u] = w; // undirected
```

Complexity

Space: $\mathcal{O}(V^2)$. Edge query: $\mathcal{O}(1)$. Use for dense graphs or Floyd-Warshall.

1.2.3 Edge List

```
1 struct Edge { int u, v, w; };
2 vector<Edge> edges;
3 edges.push_back({u, v, w});
```

Complexity

Space: $\mathcal{O}(E)$. Use for Kruskal's MST or when you need to sort edges.

	Space	Edge check	Iterate neighbours
Adjacency list	$\mathcal{O}(V + E)$	$\mathcal{O}(\text{deg})$	$\mathcal{O}(\text{deg})$
Adjacency matrix	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	$\mathcal{O}(V)$
Edge list	$\mathcal{O}(E)$	$\mathcal{O}(E)$	$\mathcal{O}(E)$

1.3 BFS — Breadth-First Search**Pattern / Core Idea**

Explores level by level. Guarantees **shortest path in unweighted graphs**.
Use a queue; mark visited *before* pushing to avoid duplicates.

```
1 vector<int> bfs(int src, int n, vector<vector<int>>& adj) {
2     vector<int> dist(n, -1);
3     queue<int> q;
4     dist[src] = 0;
5     q.push(src);
6     while (!q.empty()) {
7         int u = q.front(); q.pop();
8         for (int v : adj[u]) {
9             if (dist[v] == -1) {           // not visited
10                 dist[v] = dist[u] + 1;
11                 q.push(v);
12             }
13         }
14     }
15     return dist;                          // dist[i] = shortest hops from src to i
16 }
```

Complexity

$\mathcal{O}(V + E)$ time and space.

Common Trap / Gotcha

Mark visited when pushing, not when popping. If you mark on pop, a node may be enqueued multiple times before being processed — $\mathcal{O}(E)$ extra work in worst case.

1.4 DFS — Depth-First Search

```

1  vector<bool> visited;
2
3  void dfs(int u, vector<vector<int>>& adj) {
4      visited[u] = true;
5      for (int v : adj[u])
6          if (!visited[v]) dfs(v, adj);
7  }
8
9  // Iterative DFS (avoids stack overflow for large graphs)
10 void dfsIter(int src, vector<vector<int>>& adj) {
11     stack<int> stk;
12     stk.push(src);
13     while (!stk.empty()) {
14         int u = stk.top(); stk.pop();
15         if (visited[u]) continue;
16         visited[u] = true;
17         for (int v : adj[u]) if (!visited[v]) stk.push(v);
18     }
19 }

```

Complexity

$\mathcal{O}(V + E)$ time and space.

1.5 Connected Components & Flood Fill

```

1  int countComponents(int n, vector<vector<int>>& adj) {
2      vector<bool> vis(n, false);
3      int comps = 0;
4      for (int i = 0; i < n; i++)
5          if (!vis[i]) { dfs(i, adj); comps++; } // reuse dfs above
6      return comps;
7  }

```

1.6 Cycle Detection

1.6.1 Undirected Graph (DFS)

```

1  bool hasCycleUndirected(int u, int parent, vector<vector<int>>& adj,
2                          vector<bool>& vis) {
3      vis[u] = true;
4      for (int v : adj[u]) {
5          if (!vis[v]) {
6              if (hasCycleUndirected(v, u, adj, vis)) return true;
7          } else if (v != parent) return true; // back edge (not the tree edge)
8      }
9      return false;
10 }

```

1.6.2 Directed Graph (DFS + recursion stack)

```

1  bool hasCycleDir(int u, vector<vector<int>>& adj,
2                  vector<bool>& vis, vector<bool>& inStack) {
3      vis[u] = inStack[u] = true;
4      for (int v : adj[u]) {
5          if (!vis[v] && hasCycleDir(v, adj, vis, inStack)) return true;
6          else if (inStack[v]) return true; // back edge in directed graph
7      }
8      inStack[u] = false;
9      return false;

```

```
10 }
```

1.7 Topological Sort

1.7.1 DFS-based (Postorder)

```
1 void topoUtil(int u, vector<vector<int>>& adj,
2             vector<bool>& vis, stack<int>& stk) {
3     vis[u] = true;
4     for (int v : adj[u]) if (!vis[v]) topoUtil(v, adj, vis, stk);
5     stk.push(u);           // push AFTER all descendants
6 }
7
8 vector<int> topoSort(int n, vector<vector<int>>& adj) {
9     vector<bool> vis(n, false);
10    stack<int> stk;
11    for (int i = 0; i < n; i++) if (!vis[i]) topoUtil(i, adj, vis, stk);
12    vector<int> order;
13    while (!stk.empty()) { order.push_back(stk.top()); stk.pop(); }
14    return order;
15 }
```

1.7.2 Kahn's Algorithm (BFS-based, detects cycles)

```
1 vector<int> kahnTopoSort(int n, vector<vector<int>>& adj) {
2     vector<int> indeg(n, 0);
3     for (int u = 0; u < n; u++)
4         for (int v : adj[u]) indeg[v]++;
5
6     queue<int> q;
7     for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
8
9     vector<int> order;
10    while (!q.empty()) {
11        int u = q.front(); q.pop();
12        order.push_back(u);
13        for (int v : adj[u])
14            if (--indeg[v] == 0) q.push(v);
15    }
16    // If order.size() != n: graph has a cycle
17    return order;
18 }
```

Key Insight / Optimization

Kahn's vs DFS topo: Kahn's is iterative (no stack overflow) and naturally detects cycles (order size $< n$). DFS topo is shorter to write. Both are $\mathcal{O}(V + E)$.

1.8 Bipartite Check

```
1 bool isBipartite(int n, vector<vector<int>>& adj) {
2     vector<int> color(n, -1);
3     for (int start = 0; start < n; start++) {
4         if (color[start] != -1) continue;
5         queue<int> q;
6         color[start] = 0; q.push(start);
7         while (!q.empty()) {
8             int u = q.front(); q.pop();
9             for (int v : adj[u]) {
10                 if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
11             }
12         }
13     }
14 }
```



```
11         else if (color[v] == color[u]) return false; // odd cycle
12     }
13 }
14 return true;
15 }
16 }
```


Chapter 2

Shortest Paths

2.1 Algorithm Selector

Algorithm	Graph type	Complexity	Handles neg?
BFS	Unweighted	$\mathcal{O}(V + E)$	—
0-1 BFS	Weights $\in \{0, 1\}$	$\mathcal{O}(V + E)$	No
Dijkstra	Non-negative weights	$\mathcal{O}((V + E) \log V)$	No
Bellman-Ford	Any weights	$\mathcal{O}(VE)$	Yes
SPFA	Any weights (avg)	$\mathcal{O}(kE)$	Yes
Floyd-Warshall	All pairs, dense	$\mathcal{O}(V^3)$	Yes (no neg cycle)

2.2 Dijkstra's Algorithm

Pattern / Core Idea

Greedy SSSP for non-negative weights. Uses a min-heap to always relax the closest unvisited vertex.
Fails with negative edges (greedy assumption breaks).

```
1  const int INF = 1e9;
2  using pii = pair<int,int>;
3
4  vector<int> dijkstra(int src, int n, vector<vector<pii>>& adj) {
5      vector<int> dist(n, INF);
6      priority_queue<pii, vector<pii>, greater<pii>> pq; // min-heap {dist, node}
7      dist[src] = 0;
8      pq.push({0, src});
9
10     while (!pq.empty()) {
11         auto [d, u] = pq.top(); pq.pop();
12         if (d > dist[u]) continue; // stale entry -- skip
13         for (auto [v, w] : adj[u]) {
14             if (dist[u] + w < dist[v]) {
15                 dist[v] = dist[u] + w;
16                 pq.push({dist[v], v});
17             }
18         }
19     }
20     return dist;
21 }
```

Complexity

$\mathcal{O}((V + E) \log V)$ with binary heap. $\mathcal{O}(V^2)$ with array (better for dense graphs).

Common Trap / Gotcha

Stale entries: The priority queue may hold outdated (larger) distances for already-settled vertices. The `if (d > dist[u]) continue` guard is essential — without it, you'll reprocess nodes unnecessarily and potentially get wrong answers if using a visited array incorrectly.

2.2.1 Dijkstra with Path Reconstruction

```

1 vector<int> dijkstraPath(int src, int dst, int n, vector<vector<pii>>& adj) {
2     vector<int> dist(n, INF), parent(n, -1);
3     priority_queue<pii, vector<pii>, greater<pii>> pq;
4     dist[src] = 0; pq.push({0, src});
5     while (!pq.empty()) {
6         auto [d, u] = pq.top(); pq.pop();
7         if (d > dist[u]) continue;
8         for (auto [v, w] : adj[u]) {
9             if (dist[u] + w < dist[v]) {
10                dist[v] = dist[u] + w;
11                parent[v] = u;           // track predecessor
12                pq.push({dist[v], v});
13            }
14        }
15    }
16    vector<int> path;
17    for (int v = dst; v != -1; v = parent[v]) path.push_back(v);
18    reverse(path.begin(), path.end());
19    return path;
20 }
```

2.3 0-1 BFS**Pattern / Core Idea**

When edge weights are only 0 or 1, use a **deque** instead of a heap. Push weight-0 edges to the *front*, weight-1 edges to the *back*. This gives $\mathcal{O}(V + E)$ — same as BFS.

```

1 vector<int> zeroOneBFS(int src, int n, vector<vector<pii>>& adj) {
2     vector<int> dist(n, INF);
3     deque<int> dq;
4     dist[src] = 0; dq.push_front(src);
5
6     while (!dq.empty()) {
7         int u = dq.front(); dq.pop_front();
8         for (auto [v, w] : adj[u]) {
9             if (dist[u] + w < dist[v]) {
10                dist[v] = dist[u] + w;
11                if (w == 0) dq.push_front(v); // free edge: process first
12                else       dq.push_back(v);  // cost-1 edge: process later
13            }
14        }
15    }
16    return dist;
17 }
```

Key Insight / Optimization

Generalise to k -weight: For weights in $\{0, 1, \dots, k\}$, use a dial's algorithm with a bucket queue of size $k \cdot V$: $\mathcal{O}(kV + E)$.

2.4 Bellman-Ford

Pattern / Core Idea

Relax *all* edges $V - 1$ times. After k rounds, finds shortest paths using at most k edges. Run a V -th round: if any edge still relaxes, there is a **negative cycle**.

```

1 struct Edge { int u, v, w; };
2
3 pair<vector<int>, bool> bellmanFord(int src, int n, vector<Edge>& edges) {
4     vector<int> dist(n, INF);
5     dist[src] = 0;
6
7     for (int i = 0; i < n - 1; i++)          // relax V-1 times
8         for (auto& [u, v, w] : edges)
9             if (dist[u] != INF && dist[u] + w < dist[v])
10                 dist[v] = dist[u] + w;
11
12     bool negCycle = false;
13     for (auto& [u, v, w] : edges)          // V-th round: detect negative cycle
14         if (dist[u] != INF && dist[u] + w < dist[v]) { negCycle = true; break; }
15
16     return {dist, negCycle};
17 }
```

Complexity

$\mathcal{O}(VE)$ time. $\mathcal{O}(V)$ space.

2.4.1 SPFA — Shortest Path Faster Algorithm

Queue-optimised Bellman-Ford. Only re-relax vertices whose distance just improved.

```

1 vector<int> spfa(int src, int n, vector<vector<pii>>& adj) {
2     vector<int> dist(n, INF), inQueue(n, 0), cnt(n, 0);
3     queue<int> q;
4     dist[src] = 0; q.push(src); inQueue[src] = 1;
5
6     while (!q.empty()) {
7         int u = q.front(); q.pop(); inQueue[u] = 0;
8         for (auto [v, w] : adj[u]) {
9             if (dist[u] + w < dist[v]) {
10                 dist[v] = dist[u] + w;
11                 if (!inQueue[v]) {
12                     q.push(v); inQueue[v] = 1;
13                     if (++cnt[v] > n) return {}; // negative cycle: cnt[v] > V
14                 }
15             }
16         }
17     }
18     return dist;
19 }
```

Common Trap / Gotcha

SPFA is not reliable in contests: Its worst case is still $\mathcal{O}(VE)$ and adversarial test cases can make it degrade. Prefer Bellman-Ford when you need a correctness guarantee, and Dijkstra when weights are non-negative.

2.5 Floyd-Warshall — All-Pairs Shortest Path

Pattern / Core Idea

DP over intermediate vertices. $\text{dist}[i][j]$ = shortest path from i to j .
After k iterations, $\text{dist}[i][j]$ is shortest using only vertices $\{0, \dots, k\}$ as intermediates.

```

1  const int INF = 1e9;
2  int dist[505][505];
3
4  void floydWarshall(int n) {
5      // Initialise: dist[i][i] = 0, dist[i][j] = edge weight or INF
6      for (int k = 0; k < n; k++)
7          for (int i = 0; i < n; i++)
8              for (int j = 0; j < n; j++)
9                  if (dist[i][k] < INF && dist[k][j] < INF)
10                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
11     // After: dist[i][i] < 0 means vertex i is on a negative cycle
12 }

```

Complexity

$\mathcal{O}(V^3)$ time, $\mathcal{O}(V^2)$ space. Practical for $V \leq 500$.

Key Insight / Optimization

Transitive Closure: Use the same DP with boolean $\text{reach}[i][j]$ to check reachability in $\mathcal{O}(V^3)$:

```

1  bool reach[505][505]; // reach[i][j] = true if path i -> j exists
2  for (int k = 0; k < n; k++)
3      for (int i = 0; i < n; i++)
4          for (int j = 0; j < n; j++)
5              reach[i][j] = reach[i][k] && reach[k][j];

```

2.6 Multi-Source BFS

Push *all* sources at distance 0 simultaneously. Each cell gets the distance to its nearest source.

```

1  // Classic use: grid problem -- distance to nearest '1' (LeetCode 542)
2  vector<vector<int>> multiSourceBFS(vector<vector<int>>& grid) {
3      int R = grid.size(), C = grid[0].size();
4      vector<vector<int>> dist(R, vector<int>(C, INT_MAX));
5      queue<pair<int,int>> q;
6      int dr[] = {0,0,1,-1}, dc[] = {1,-1,0,0};
7
8      for (int r = 0; r < R; r++)
9          for (int c = 0; c < C; c++)
10             if (grid[r][c] == 1) { dist[r][c] = 0; q.push({r, c}); }
11
12     while (!q.empty()) {
13         auto [r, c] = q.front(); q.pop();

```

```

14     for (int d = 0; d < 4; d++) {
15         int nr = r+dr[d], nc = c+dc[d];
16         if (nr>=0 && nr<R && nc>=0 && nc<C && dist[nr][nc]==INT_MAX) {
17             dist[nr][nc] = dist[r][c] + 1;
18             q.push({nr, nc});
19         }
20     }
21 }
22 return dist;
23 }

```

2.7 Shortest Path in a DAG (Topological Order DP)

```

1  // For DAGs: relax edges in topological order -- O(V+E), handles negative weights
2  vector<int> dagSSSP(int src, int n, vector<vector<pii>>& adj) {
3      vector<int> order = kahnTopoSort(n, adj); // from ch1
4      vector<int> dist(n, INF);
5      dist[src] = 0;
6      for (int u : order)
7          for (auto [v, w] : adj[u])
8              if (dist[u] != INF)
9                  dist[v] = min(dist[v], dist[u] + w);
10     return dist;
11 }

```

Key Insight / Optimization

Longest path in a DAG: Negate all weights, run DAG SSSP, negate the result. Equivalently, change `min` to `max` and initialise `dist[src]=0`, rest to `-INF`. This solves the **critical path** problem in project scheduling.

Chapter 3

Minimum Spanning Tree

3.1 MST Fundamentals

Pattern / Core Idea

A **Minimum Spanning Tree** of a connected undirected weighted graph is a spanning tree (connects all V vertices, uses $V - 1$ edges) with minimum total edge weight. **Cut property:** For any cut of the graph, the minimum-weight crossing edge is always in some MST. **Cycle property:** The maximum-weight edge in any cycle is never in the MST.

3.2 DSU — Disjoint Set Union (Union-Find)

DSU is the backbone of Kruskal's. Essential to know cold.

```
1 struct DSU {
2     vector<int> parent, rank_;
3     DSU(int n) : parent(n), rank_(n, 0) { iota(parent.begin(), parent.end(), 0); }
4
5     int find(int x) {                                     // path compression
6         if (parent[x] != x) parent[x] = find(parent[x]);
7         return parent[x];
8     }
9     bool unite(int x, int y) {                             // union by rank
10        x = find(x); y = find(y);
11        if (x == y) return false;                          // same component: cycle!
12        if (rank_[x] < rank_[y]) swap(x, y);
13        parent[y] = x;
14        if (rank_[x] == rank_[y]) rank_[x]++;
15        return true;
16    }
17    bool connected(int x, int y) { return find(x) == find(y); }
18 };
```

Complexity

With path compression + union by rank: $\mathcal{O}(\alpha(n))$ per operation (effectively $\mathcal{O}(1)$).

3.3 Kruskal's Algorithm

Pattern / Core Idea

Sort all edges by weight. Greedily add the cheapest edge that doesn't form a cycle (checked with DSU).

```

1 struct Edge { int u, v, w; };
2
3 int kruskal(int n, vector<Edge>& edges) {
4     sort(edges.begin(), edges.end(), [](auto& a, auto& b){ return a.w < b.w; });
5     DSU dsu(n);
6     int totalCost = 0, edgesUsed = 0;
7     for (auto& [u, v, w] : edges) {
8         if (dsu.unite(u, v)) { // no cycle: add edge
9             totalCost += w;
10            if (++edgesUsed == n - 1) break; // MST complete
11        }
12    }
13    return totalCost; // -1 if graph is disconnected
14 }
```

Complexity

$\mathcal{O}(E \log E)$ dominated by sorting. $\mathcal{O}(E\alpha(V))$ for DSU operations.

When to use

Use Kruskal when: $E \ll V^2$ (sparse), or you have an edge list ready, or you need to process edges in weight order for other reasons.

3.4 Prim's Algorithm

Pattern / Core Idea

Grow the MST one vertex at a time. Always add the cheapest edge connecting the current tree to a new vertex. Uses a min-heap like Dijkstra.

```

1 using pii = pair<int,int>;
2
3 int prim(int n, vector<vector<pii>>& adj) {
4     vector<int> key(n, INT_MAX);
5     vector<bool> inMST(n, false);
6     priority_queue<pii, vector<pii>, greater<pii>> pq;
7     key[0] = 0; pq.push({0, 0});
8     int totalCost = 0;
9
10    while (!pq.empty()) {
11        auto [w, u] = pq.top(); pq.pop();
12        if (inMST[u]) continue;
13        inMST[u] = true;
14        totalCost += w;
15        for (auto [v, wt] : adj[u]) {
16            if (!inMST[v] && wt < key[v]) {
17                key[v] = wt;
18                pq.push({wt, v});
19            }
20        }
21    }
22    return totalCost;
23 }
```

Complexity

$\mathcal{O}((V + E) \log V)$ with binary heap. $\mathcal{O}(V^2)$ with simple array (better for dense graphs).

When to use

Use Prim when: graph is dense ($E \approx V^2$), you want a vertex-growing approach, or you're building the MST incrementally.

3.5 Second Minimum Spanning Tree

```

1 // Find the MST, then for each non-tree edge (u,v,w):
2 // the second MST is obtained by adding (u,v,w) and removing the max edge on path u->v in
  // the MST.
3 // Precompute max edge on any tree path using LCA + sparse table (see Ch 6).
4 // Simple  $\mathcal{O}(V \cdot E)$  approach: for each non-tree edge, find max edge on path via DFS.
5
6 int secondMST(int n, vector<Edge>& edges, vector<Edge>& mstEdges) {
7     int mstCost = 0;
8     for (auto& e : mstEdges) mstCost += e.w;
9
10    // Build MST adjacency list
11    vector<vector<pii>> tree(n);
12    for (auto& [u, v, w] : mstEdges) {
13        tree[u].push_back({v, w});
14        tree[v].push_back({u, w});
15    }
16    // For each non-MST edge (u,v,w): find max edge weight on path u->v in tree
17    int best = INT_MAX;
18    for (auto& [u, v, w] : edges) {
19        // DFS to find maxEdge on path u->v in the tree
20        vector<int> maxEdge(n, -1); vector<bool> vis(n, false);
21        function<bool(int, int)> dfs = [&](int cur, int par) -> bool {
22            if (cur == v) return true;
23            vis[cur] = true;
24            for (auto [nxt, wt] : tree[cur]) {
25                if (vis[nxt] || nxt == par) continue;
26                maxEdge[nxt] = max(wt, maxEdge[cur] == -1 ? wt : maxEdge[cur]);
27                if (dfs(nxt, cur)) return true;
28            }
29            return false;
30        };
31        maxEdge[u] = -1; fill(vis.begin(), vis.end(), false);
32        if (dfs(u, -1) && maxEdge[v] != -1 && w != maxEdge[v])
33            best = min(best, mstCost - maxEdge[v] + w);
34    }
35    return best; // second minimum spanning tree cost
36 }
```

3.6 Minimum Spanning Forest

For disconnected graphs, run Kruskal/Prim normally — the algorithm naturally produces a spanning forest (one tree per component).

```

1 // Kruskal unchanged: returns sum of MST edges, components that stay
2 // disconnected simply aren't joined.
3 // To count components: edgesUsed < n-1 means graph is disconnected.
4 int components = n - edgesUsed; // number of connected components
```

3.7 Borůvka's Algorithm (Non-Standard)

Pattern / Core Idea

Each component picks its cheapest outgoing edge simultaneously. Repeat until one component remains. $\mathcal{O}(E \log V)$ but parallelisable — basis of some distributed MST algorithms.

```

1 int boruvka(int n, vector<Edge>& edges) {
2     DSU dsu(n);
3     int totalCost = 0, components = n;
4
5     while (components > 1) {
6         vector<int> cheapest(n, -1);           // cheapest edge index per component
7         for (int i = 0; i < (int)edges.size(); i++) {
8             auto& [u, v, w] = edges[i];
9             int cu = dsu.find(u), cv = dsu.find(v);
10            if (cu == cv) continue;           // same component
11            if (cheapest[cu]==-1 || edges[cheapest[cu]].w > w) cheapest[cu]=i;
12            if (cheapest[cv]==-1 || edges[cheapest[cv]].w > w) cheapest[cv]=i;
13        }
14        for (int i = 0; i < n; i++) {
15            if (cheapest[i] == -1) continue;
16            auto& [u, v, w] = edges[cheapest[i]];
17            if (dsu.unite(u, v)) { totalCost += w; components--; }
18        }
19    }
20    return totalCost;
21 }
```

Complexity

$\mathcal{O}(E \log V)$: $\mathcal{O}(\log V)$ phases, each scanning all E edges.

Chapter 4

SCCs, Bridges & Articulation Points

4.1 Strongly Connected Components (SCCs)

Pattern / Core Idea

An SCC of a directed graph is a maximal set of vertices such that there is a path from every vertex to every other. The SCC DAG (condensation) is always a DAG.

4.2 Kosaraju's Algorithm

Two-pass DFS. First pass: record finish order on original graph. Second pass: DFS on reversed graph in reverse finish order.

```
1 int n;
2 vector<int> adj[100005], radj[100005]; // original and reversed graph
3 vector<bool> vis;
4 vector<int> order;
5 int comp[100005];
6
7 void dfs1(int u) {
8     vis[u] = true;
9     for (int v : adj[u]) if (!vis[v]) dfs1(v);
10    order.push_back(u); // push on finish
11 }
12
13 void dfs2(int u, int c) {
14     comp[u] = c;
15     for (int v : radj[u]) if (comp[v] == -1) dfs2(v, c);
16 }
17
18 int kosaraju() {
19     vis.assign(n, false);
20     for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
21     fill(comp, comp + n, -1);
22     int numSCC = 0;
23     for (int i = n - 1; i >= 0; i--) // process in reverse finish order
24         if (comp[order[i]] == -1) dfs2(order[i], numSCC++);
25     return numSCC;
26 }
```

Complexity

$\mathcal{O}(V + E)$: two full DFS traversals.

4.3 Tarjan's Algorithm for SCCs

Single-pass DFS using discovery time and low-link values. Also finds bridges and articulation points.

```

1  int timer_t = 0;
2  vector<int> disc, low, sccId;
3  vector<bool> onStack;
4  stack<int> stk;
5  int numSCC = 0;
6
7  void tarjanSCC(int u, vector<vector<int>>& adj) {
8      disc[u] = low[u] = timer_t++;
9      stk.push(u); onStack[u] = true;
10
11     for (int v : adj[u]) {
12         if (disc[v] == -1) {                // tree edge
13             tarjanSCC(v, adj);
14             low[u] = min(low[u], low[v]);
15         } else if (onStack[v]) {            // back edge to ancestor in stack
16             low[u] = min(low[u], disc[v]);
17         }
18     }
19     // u is the root of an SCC if low[u] == disc[u]
20     if (low[u] == disc[u]) {
21         while (true) {
22             int v = stk.top(); stk.pop();
23             onStack[v] = false;
24             sccId[v] = numSCC;
25             if (v == u) break;
26         }
27         numSCC++;
28     }
29 }
30
31 void findSCCs(int n, vector<vector<int>>& adj) {
32     disc.assign(n, -1); low.assign(n, 0);
33     sccId.assign(n, -1); onStack.assign(n, false);
34     for (int i = 0; i < n; i++) if (disc[i] == -1) tarjanSCC(i, adj);
35 }

```

Key Insight / Optimization

Tarjan vs Kosaraju: Tarjan is one-pass (faster constant), handles SCCs in reverse topological order of the condensation, and generalises to find bridges and articulation points. Kosaraju is easier to remember and implement in contests.

4.4 Bridges

Pattern / Core Idea

An edge (u, v) is a **bridge** if removing it increases the number of connected components. Equivalently: it's a bridge iff $\text{low}[v] > \text{disc}[u]$ (no back edge from v 's subtree can reach u or above).

```

1  int timer_b = 0;
2  vector<int> disc_b, low_b;
3  vector<pair<int, int>> bridges;
4
5  void dfs_bridge(int u, int parent, vector<vector<int>>& adj) {
6      disc_b[u] = low_b[u] = timer_b++;
7      for (int v : adj[u]) {
8          if (disc_b[v] == -1) {

```

```

9         dfs_bridge(v, u, adj);
10        low_b[u] = min(low_b[u], low_b[v]);
11        if (low_b[v] > disc_b[u])           // bridge condition
12            bridges.push_back({u, v});
13    } else if (v != parent) {               // back edge (not tree edge)
14        low_b[u] = min(low_b[u], disc_b[v]);
15    }
16 }
17 }

```

Common Trap / Gotcha

Multi-edges (parallel edges): If (u, v) has two parallel edges, neither is a bridge. Track edge index instead of parent vertex: if $(\text{edgeIdx} \neq \text{parentEdgeIdx})$ instead of if $(v \neq \text{parent})$.

```

1 // Adj stored as {neighbour, edgeIndex}
2 void dfs_bridge_safe(int u, int parentEdge, vector<vector<pii>>& adj) {
3     disc_b[u] = low_b[u] = timer_b++;
4     for (auto [v, eid] : adj[u]) {
5         if (disc_b[v] == -1) {
6             dfs_bridge_safe(v, eid, adj);
7             low_b[u] = min(low_b[u], low_b[v]);
8             if (low_b[v] > disc_b[u]) bridges.push_back({u, v});
9         } else if (eid != parentEdge)
10            low_b[u] = min(low_b[u], disc_b[v]);
11    }
12 }

```

4.5 Articulation Points (Cut Vertices)

Pattern / Core Idea

A vertex u is an **articulation point** if removing it splits the graph. Two conditions:

1. u is the DFS root and has ≥ 2 children in the DFS tree.
2. u is not the root and has a child v with $\text{low}[v] \geq \text{disc}[u]$ (no back edge from v 's subtree reaches above u).

```

1 vector<bool> isAP;
2 int timer_ap = 0;
3 vector<int> disc_ap, low_ap;
4
5 void dfs_ap(int u, int parent, vector<vector<int>>& adj) {
6     disc_ap[u] = low_ap[u] = timer_ap++;
7     int children = 0;
8     for (int v : adj[u]) {
9         if (disc_ap[v] == -1) {
10            children++;
11            dfs_ap(v, u, adj);
12            low_ap[u] = min(low_ap[u], low_ap[v]);
13            // AP condition 2: non-root with a child that can't reach above u
14            if (parent != -1 && low_ap[v] >= disc_ap[u]) isAP[u] = true;
15        } else if (v != parent) {
16            low_ap[u] = min(low_ap[u], disc_ap[v]);
17        }
18    }
19    // AP condition 1: root with multiple DFS children
20    if (parent == -1 && children > 1) isAP[u] = true;
21 }

```

4.6 Bridge Tree (Block-Cut Tree)

After finding all bridges and articulation points, the graph can be compressed into a tree where:

- Each 2-edge-connected component becomes a single node.
- Bridges become edges between these nodes.

Queries on paths in the original graph (e.g., “how many bridges on path $u \rightarrow v$?”) reduce to LCA queries on the bridge tree.

```

1 // 1. Find all bridges
2 // 2. Assign component IDs via DFS, skipping bridge edges
3 // 3. Build bridge tree: new graph where compId[u] connects to compId[v] via bridge (u,v)
4 vector<int> compId(n, -1);
5 int numComp = 0;
6
7 void assignComp(int u, int id, vector<vector<int>>& adj,
8               set<pair<int,int>>& bridgeSet) {
9     compId[u] = id;
10    for (int v : adj[u]) {
11        if (compId[v] == -1 && !bridgeSet.count({min(u,v), max(u,v)}))
12            assignComp(v, id, adj, bridgeSet);
13    }
14 }
15 // Bridge tree has numComp nodes and |bridges| edges

```

4.7 Eulerian Path and Circuit

Pattern / Core Idea

Euler circuit (visits every edge exactly once, returns to start):

Undirected: every vertex has even degree, graph is connected.

Directed: every vertex has $\deg^+ = \deg^-$, graph is strongly connected.

Euler path (visits every edge exactly once):

Undirected: exactly 2 vertices have odd degree (start and end).

Directed: one vertex has $\deg^+ - \deg^- = 1$ (start), one has $\deg^- - \deg^+ = 1$ (end).

```

1 // Hierholzer's algorithm: O(E)
2 vector<int> eulerCircuit(int start, int n, vector<multiset<int>>& adj) {
3     stack<int> stk;
4     vector<int> circuit;
5     stk.push(start);
6     while (!stk.empty()) {
7         int u = stk.top();
8         if (adj[u].empty()) {
9             circuit.push_back(u);    // dead end: add to circuit
10            stk.pop();
11        } else {
12            int v = *adj[u].begin();
13            adj[u].erase(adj[u].begin());
14            adj[v].erase(adj[v].find(u)); // remove reverse edge (undirected)
15            stk.push(v);
16        }
17    }
18    reverse(circuit.begin(), circuit.end());
19    return circuit; // size == E+1 if Eulerian circuit exists
20 }

```


Chapter 5

Network Flow

5.1 Flow Network Basics

Pattern / Core Idea

A **flow network**: directed graph with a *source* s , *sink* t , and capacity $c(u, v) \geq 0$ on each edge. A *flow* $f(u, v)$ satisfies:

- **Capacity**: $0 \leq f(u, v) \leq c(u, v)$
- **Conservation**: flow into u = flow out of u for all $u \neq s, t$

Max-flow min-cut theorem: Maximum flow = minimum s - t cut capacity.

5.2 Ford-Fulkerson / Edmonds-Karp

```
1 struct MaxFlow {
2     struct Edge { int to, rev; int cap; };
3     int n;
4     vector<vector<Edge>> graph;
5     vector<int> level, iter_;
6
7     MaxFlow(int n) : n(n), graph(n), level(n), iter_(n) {}
8
9     void addEdge(int from, int to, int cap) {
10         graph[from].push_back({to, (int)graph[to].size(), cap});
11         graph[to].push_back({from, (int)graph[from].size()-1, 0}); // reverse edge
12     }
13
14     bool bfs(int s, int t) { // Edmonds-Karp: BFS finds shortest augmenting path
15         fill(level.begin(), level.end(), -1);
16         queue<int> q;
17         level[s] = 0; q.push(s);
18         while (!q.empty()) {
19             int v = q.front(); q.pop();
20             for (auto& e : graph[v])
21                 if (e.cap > 0 && level[e.to] < 0) {
22                     level[e.to] = level[v] + 1; q.push(e.to);
23                 }
24         }
25         return level[t] >= 0;
26     }
27
28     int dfs(int v, int t, int f) {
29         if (v == t) return f;
30         for (int& i = iter_[v]; i < (int)graph[v].size(); i++) {
31             Edge& e = graph[v][i];
32             if (e.cap > 0 && level[v] < level[e.to]) {
33                 int d = dfs(e.to, t, min(f, e.cap));
```

```

34         if (d > 0) {
35             e.cap -= d;
36             graph[e.to][e.rev].cap += d;
37             return d;
38         }
39     }
40 }
41 return 0;
42 }
43
44 // Dinic's algorithm:  $O(V^2 * E)$ ,  $O(E * \sqrt{V})$  for unit-capacity
45 long long maxflow(int s, int t) {
46     long long flow = 0;
47     while (bfs(s, t)) {
48         fill(iter_.begin(), iter_.end(), 0);
49         int d;
50         while ((d = dfs(s, t, INT_MAX)) > 0) flow += d;
51     }
52     return flow;
53 }
54 };

```

Complexity

Dinic's: $O(V^2E)$ general. $O(E\sqrt{V})$ on unit-capacity graphs (bipartite matching). $O(E\sqrt{E})$ on unit-capacity networks.

Edmonds-Karp (BFS augmenting): $O(VE^2)$.

Key Insight / Optimization

The reverse edge trick: Every edge (u, v, c) gets a paired reverse edge $(v, u, 0)$. Pushing flow along (u, v) reduces its capacity and increases the reverse edge capacity — this allows “undoing” bad flow decisions along augmenting paths.

5.3 Min-Cut from Max-Flow

After computing max flow, the min cut is found by BFS/DFS on the residual graph from s :

```

1 // After maxflow(), BFS from s on residual (edges with cap > 0)
2 // S-side = vertices reachable from s in residual
3 // T-side = everything else
4 // Cut edges = original edges (u,v) with u in S-side, v in T-side
5 vector<bool> minCutSide(int s) {
6     vector<bool> visited(n, false);
7     queue<int> q;
8     visited[s] = true; q.push(s);
9     while (!q.empty()) {
10         int u = q.front(); q.pop();
11         for (auto& e : graph[u])
12             if (e.cap > 0 && !visited[e.to]) {
13                 visited[e.to] = true; q.push(e.to);
14             }
15     }
16     return visited; // S-side vertices
17 }

```

5.4 Bipartite Matching

Pattern / Core Idea

Model as flow: source $\rightarrow$ left vertices (capacity 1) $\rightarrow$ right vertices (capacity 1) $\rightarrow$ sink. Max flow = max matching. But the simpler **Hungarian** / **Hopcroft-Karp** algorithm is faster in practice.

5.4.1 Simple DFS Matching (Augmenting Paths)

```

1  int matchL[1005], matchR[1005]; // matchL[u] = partner of left vertex u
2  bool visited_m[1005];
3  vector<int> adjL[1005];          // left adj list
4
5  bool dfs_match(int u) {
6      for (int v : adjL[u]) {
7          if (visited_m[v]) continue;
8          visited_m[v] = true;
9          if (matchR[v] == -1 || dfs_match(matchR[v])) { // augmenting path found
10             matchL[u] = v; matchR[v] = u; return true;
11         }
12     }
13     return false;
14 }
15
16 int maxBipartiteMatching(int leftN) {
17     fill(matchL, matchL + leftN, -1);
18     fill(matchR, matchR + 1005, -1);
19     int matching = 0;
20     for (int u = 0; u < leftN; u++) {
21         fill(visited_m, visited_m + 1005, false);
22         if (dfs_match(u)) matching++;
23     }
24     return matching;
25 }

```

Complexity

$\mathcal{O}(VE)$ simple DFS matching. $\mathcal{O}(E\sqrt{V})$ Hopcroft-Karp (see below).

5.4.2 Hopcroft-Karp ($\mathcal{O}(E\sqrt{V})$)

```

1  struct HopcroftKarp {
2      int L, R;
3      vector<vector<int>> adj;
4      vector<int> matchL, matchR, dist_hk;
5
6      HopcroftKarp(int L, int R) : L(L), R(R), adj(L),
7          matchL(L, -1), matchR(R, -1), dist_hk(L) {}
8
9      void addEdge(int u, int v) { adj[u].push_back(v); }
10
11     bool bfs_hk() {
12         queue<int> q;
13         for (int u = 0; u < L; u++) {
14             if (matchL[u] == -1) { dist_hk[u] = 0; q.push(u); }
15             else dist_hk[u] = INT_MAX;
16         }
17         bool found = false;
18         while (!q.empty()) {
19             int u = q.front(); q.pop();
20             for (int v : adj[u]) {
21                 int w = matchR[v];
22                 if (w == -1) { found = true; }

```

```

23         else if (dist_hk[w] == INT_MAX) {
24             dist_hk[w] = dist_hk[u] + 1; q.push(w);
25         }
26     }
27 }
28 return found;
29 }
30 bool dfs_hk(int u) {
31     for (int v : adj[u]) {
32         int w = matchR[v];
33         if (w == -1 || (dist_hk[w] == dist_hk[u]+1 && dfs_hk(w))) {
34             matchL[u] = v; matchR[v] = u; return true;
35         }
36     }
37     dist_hk[u] = INT_MAX; return false;
38 }
39 int maxMatching() {
40     int match = 0;
41     while (bfs_hk()) for (int u = 0; u < L; u++) if (matchL[u]==-1 && dfs_hk(u)) match
42 ++;
43     return match;
44 }
45 };

```

5.5 Min-Cost Max-Flow (MCMF)

Find the maximum flow with minimum total cost (each edge has capacity and cost).

```

1 struct MCMF {
2     struct Edge { int to, cap, cost, rev; };
3     int n;
4     vector<vector<Edge>> graph;
5     MCMF(int n) : n(n), graph(n) {}
6
7     void addEdge(int u, int v, int cap, int cost) {
8         graph[u].push_back({v, cap, cost, (int)graph[v].size()});
9         graph[v].push_back({u, 0, -cost, (int)graph[u].size()-1});
10    }
11
12    pair<int,int> minCostFlow(int s, int t, int maxFlow) {
13        int flow = 0, cost = 0;
14        while (flow < maxFlow) {
15            // SPFA to find shortest (minimum cost) augmenting path
16            vector<int> dist(n, INT_MAX), inq(n, 0), prevv(n,-1), preve(n,-1);
17            dist[s] = 0;
18            queue<int> q; q.push(s); inq[s] = 1;
19            while (!q.empty()) {
20                int v = q.front(); q.pop(); inq[v] = 0;
21                for (int i = 0; i < (int)graph[v].size(); i++) {
22                    auto& e = graph[v][i];
23                    if (e.cap > 0 && dist[v]+e.cost < dist[e.to]) {
24                        dist[e.to] = dist[v]+e.cost;
25                        prevv[e.to] = v; preve[e.to] = i;
26                        if (!inq[e.to]) { inq[e.to]=1; q.push(e.to); }
27                    }
28                }
29            }
30            if (dist[t] == INT_MAX) break;
31            // Find bottleneck
32            int d = maxFlow - flow;
33            for (int v = t; v != s; v = prevv[v])
34                d = min(d, graph[prevv[v]][preve[v]].cap);
35            // Augment
36            for (int v = t; v != s; v = prevv[v]) {
37                auto& e = graph[prevv[v]][preve[v]];

```

```

38         e.cap -= d; graph[v][e.rev].cap += d;
39     }
40     flow += d; cost += d * dist[t];
41 }
42 return {flow, cost};
43 }
44 };

```

Complexity

$\mathcal{O}(VEF)$ where F is the max flow value (each SPFA is $\mathcal{O}(VE)$, runs F times in worst case). Use **Johnson's potentials** / **Dijkstra with potentials** for non-negative cost edges to get $\mathcal{O}(F \cdot E \log V)$.

5.6 König's Theorem

Pattern / Core Idea

In a bipartite graph:

$$\text{Max Matching} = \text{Min Vertex Cover}$$

$$\text{Max Independent Set} = V - \text{Max Matching}$$

To find the minimum vertex cover from the matching: run the alternating path construction from unmatched left vertices.

Chapter 6

Advanced Graph Algorithms

6.1 LCA — Lowest Common Ancestor

When to use

Why you need it:

Many tree problems reduce to: “what is the relationship between two nodes?” The LCA anchors every path query on a tree — path from u to $v = \text{path}(u, \text{LCA}) + \text{path}(v, \text{LCA})$.

Without LCA: computing the path between any two nodes takes $\mathcal{O}(n)$ per query.

With binary lifting LCA: $\mathcal{O}(\log n)$ **per query after** $\mathcal{O}(n \log n)$ **preprocessing**.

Appears in: HLD, centroid decomp, virtual trees, path max/sum queries, tree distances.

6.1.1 Binary Lifting

Pattern / Core Idea

Precompute $\text{anc}[v][k] = 2^k$ -th ancestor of v . To find $\text{LCA}(u, v)$: bring the deeper node to the same depth using binary jumps, then jump both simultaneously until they meet.

```
1  const int LOG = 18;    // 2^18 > 262144; increase for larger trees
2  int anc[100005][LOG], dep[100005];
3  vector<int> tree[100005];
4
5  void dfs_lca(int u, int p, int d) {
6      anc[u][0] = p; dep[u] = d;
7      for (int k = 1; k < LOG; k++)
8          anc[u][k] = anc[anc[u][k-1]][k-1]; // 2^k-th = 2^(k-1)-th of 2^(k-1)-th
9      for (int v : tree[u]) if (v != p) dfs_lca(v, u, d + 1);
10 }
11
12 int lca(int u, int v) {
13     if (dep[u] < dep[v]) swap(u, v);
14     int diff = dep[u] - dep[v];
15     for (int k = 0; k < LOG; k++)          // lift deeper node up
16         if ((diff >> k) & 1) u = anc[u][k];
17     if (u == v) return u;                  // one is ancestor of the other
18     for (int k = LOG-1; k >= 0; k--)      // lift both until just below LCA
19         if (anc[u][k] != anc[v][k]) { u = anc[u][k]; v = anc[v][k]; }
20     return anc[u][0];
21 }
```

Complexity

Preprocessing: $\mathcal{O}(n \log n)$. Query: $\mathcal{O}(\log n)$. Space: $\mathcal{O}(n \log n)$.

6.1.2 LCA + Path Max-Edge Queries

Extend the binary lifting table to carry extra information (max edge, min edge, sum, XOR) along each ancestor jump.

```

1 int maxE[100005][LOG]; // maxE[v][k] = max edge weight on path to 2^k-th ancestor
2 vector<pair<int,int>> tree_w[100005]; // {neighbor, weight}
3
4 void dfs_lca_ext(int u, int p, int d, int ew) {
5     anc[u][0] = p; dep[u] = d; maxE[u][0] = ew;
6     for (int k = 1; k < LOG; k++) {
7         anc[u][k] = anc[anc[u][k-1]][k-1];
8         maxE[u][k] = max(maxE[u][k-1], maxE[anc[u][k-1]][k-1]);
9     }
10    for (auto [v, w] : tree_w[u]) if (v != p) dfs_lca_ext(v, u, d+1, w);
11 }
12
13 int pathMax(int u, int v) {
14     int res = 0;
15     if (dep[u] < dep[v]) swap(u, v);
16     int diff = dep[u] - dep[v];
17     for (int k = 0; k < LOG; k++) if ((diff >> k) & 1) { res = max(res, maxE[u][k]); u = anc[u][k]; }
18     if (u == v) return res;
19     for (int k = LOG-1; k >= 0; k--)
20         if (anc[u][k] != anc[v][k]) {
21             res = max({res, maxE[u][k], maxE[v][k]});
22             u = anc[u][k]; v = anc[v][k];
23         }
24     return max({res, maxE[u][0], maxE[v][0]});
25 }

```

Key Insight / Optimization

Application — Minimum Bottleneck Path: Build the MST (Kruskal), then query `pathMax(u, v)` on the MST. Result = minimum possible maximum edge on any path from u to v in the original graph. $\mathcal{O}(\log n)$ per query.

6.2 Euler Tour (DFS In/Out Times)

When to use

Why you need it:

A DFS assigns each node an entry time (`tin`) and exit time (`tout`). This **flattens a tree into an array**: the subtree of node u occupies the contiguous range `[tin[u], tout[u]]`.

This lets you use any **1D range data structure** (segment tree, BIT, sparse table) directly on subtrees.

Appears in: subtree sum/max queries, HLD, centroid decomp, virtual tree.

```

1 int tin[100005], tout[100005], order[100005], timer_et = 0;
2
3 void dfs_euler(int u, int p) {
4     tin[u] = timer_et;
5     order[timer_et++] = u; // node u enters at position tin[u]
6     for (int v : tree[u]) if (v != p) dfs_euler(v, u);
7     tout[u] = timer_et - 1; // subtree of u spans [tin[u], tout[u]]
8 }

```



```

9 // Is v in the subtree of u?
10 bool isAncestor(int u, int v) { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
11
12 // After dfs_euler, build a segment/BIT on the array 'order'.
13 // Subtree query on u = range query on [tin[u], tout[u]].
14
15 // Example: subtree sum using a BIT
16 long long bit[200005]; int N;
17 void update(int i, int val) { for (i++; i<=N; i+=i&-i) bit[i]+=val; }
18 long long query(int i) { long long s=0; for(i++;i>0;i-=i&-i) s+=bit[i]; return s; }
19 long long queryRange(int l,int r) { return query(r)-query(l-1); }
20
21 // Subtree sum of u:
22 long long subtreeSum(int u) { return queryRange(tin[u], tout[u]); }

```

6.3 Heavy-Light Decomposition (HLD)

When to use

Why you need it:

You have a tree. You want to **update or query values on a path** $u \rightarrow v$ efficiently.

Naive: walk the path $\mathcal{O}(n)$ per query. With HLD + segment tree: $\mathcal{O}(\log^2 n)$.

Core insight: Decompose the tree into *heavy chains* (always go to the child with the largest subtree). Any root-to-leaf path crosses at most $\mathcal{O}(\log n)$ light edges (chain changes), because each light edge at least halves the remaining subtree size. Each chain is a contiguous range in the DFS order segment tree query.

Appears in: Path sum/max/update on trees. The workhorse of hard tree problems.

```

1 int par[100005], heavy[100005], head_hld[100005], pos_hld[100005];
2 int dep_hld[100005], sz_hld[100005], cur_hld = 0;
3
4 int dfs_sz(int v, int p, int d) {
5     par[v] = p; dep_hld[v] = d; sz_hld[v] = 1;
6     int maxSz = 0; heavy[v] = -1;
7     for (int u : tree[v]) {
8         if (u == p) continue;
9         sz_hld[v] += dfs_sz(u, v, d+1);
10        if (sz_hld[u] > maxSz) { maxSz = sz_hld[u]; heavy[v] = u; } // heavy child
11    }
12    return sz_hld[v];
13 }
14
15 void decompose_hld(int v, int h) {
16     head_hld[v] = h; // head of current chain
17     pos_hld[v] = cur_hld++; // position in linearised array
18     if (heavy[v] != -1) decompose_hld(heavy[v], h); // extend chain
19     for (int u : tree[v])
20         if (u != par[v] && u != heavy[v])
21             decompose_hld(u, u); // start new chain
22 }
23
24 // Segment tree (point update, range sum) -- replace with your preferred segtree
25 long long seg_hld[400005];
26 void seg_update(int node, int s, int e, int i, long long v) { if (s==e) { seg_hld[node]=v; return; } int
27     m=(s+e)/2; if (i<=m) seg_update(2*node, s, m, i, v); else seg_update(2*node+1, m+1, e, i, v); seg_hld
28     [node]=seg_hld[2*node]+seg_hld[2*node+1]; }
29 long long seg_query(int node, int s, int e, int l, int r) { if (r<s || e<l) return 0; if (l<=s && e<=r)
30     return seg_hld[node]; int m=(s+e)/2; return seg_query(2*node, s, m, l, r) + seg_query(2*node+1, m
31     +1, e, l, r); }
32
33 // Query sum on path u->v

```

```

30 long long pathQuery(int u, int v, int n) {
31     long long res = 0;
32     while (head_hld[u] != head_hld[v]) {
33         if (dep_hld[head_hld[u]] < dep_hld[head_hld[v]]) swap(u, v);
34         res += seg_query(1, 0, n-1, pos_hld[u], pos_hld[v]);
35         u = par[head_hld[u]]; // jump to parent of chain head
36     }
37     if (dep_hld[u] > dep_hld[v]) swap(u, v);
38     res += seg_query(1, 0, n-1, pos_hld[u], pos_hld[v]); // LCA included
39     return res;
40 }

```

Complexity

Preprocessing: $\mathcal{O}(n)$. Path query/update: $\mathcal{O}(\log^2 n)$. Can be $\mathcal{O}(\log n)$ with a Fenwick tree.

Common Trap / Gotcha

Edge weights vs vertex weights: HLD naturally handles vertex weights. For edge weights, assign the edge weight to the *lower* endpoint (the child), then exclude the LCA from the path query (query $[\text{pos}[u], \text{pos}[v]]$ with LCA's position excluded by querying $[\text{pos}[\text{child of LCA}], \text{pos}[v]]$).

6.4 Centroid Decomposition

When to use

Why you need it:

You want to **count/query paths that pass through every vertex of the tree** efficiently.

Naive: for each pair (u, v) , walk the path $\mathcal{O}(n^2)$ total.

With centroid decomp: $\mathcal{O}(n \log n)$ to process *all paths*.

Core insight: The centroid of any subtree splits it into parts each $\leq n/2$ nodes. Every path either passes through the centroid or lies entirely within one of its subtrees. This gives a *divide and conquer* on the tree with $\mathcal{O}(\log n)$ levels.

Appears in: Count pairs of nodes with path length = k , path XOR queries, k-closest nodes.

```

1  int sz_cd[100005];
2  bool removed[100005];
3  int centPar[100005];
4
5  void calcSize(int u, int p) {
6      sz_cd[u] = 1;
7      for (int v : tree[u]) if (v != p && !removed[v])
8          calcSize(v, u), sz_cd[u] += sz_cd[v];
9  }
10
11 int getCentroid(int u, int p, int treeN) {
12     for (int v : tree[u]) if (v != p && !removed[v])
13         if (sz_cd[v] > treeN / 2) return getCentroid(v, u, treeN);
14     return u;
15 }
16
17 // Typical centroid decomp template:
18 // For each centroid c, collect distances from c to all nodes in its subtree,
19 // then count pairs using a hash map or sorted array.
20 void solve(int u) {
21     calcSize(u, -1);
22     int c = getCentroid(u, -1, sz_cd[u]);
23     removed[c] = true;
24     // --- process all paths through centroid c ---

```

```

25 // e.g., gather dist[] from c, count pairs with dist[a]+dist[b] == k
26 // using a frequency map: for each node v at dist d, check map[k-d]
27 // -----
28 for (int v : tree[c]) if (!removed[v]) solve(v); // recurse on subtrees
29 }

```

```

1 // Concrete example: count pairs of nodes with path length exactly K
2 long long countPairs(int u, int parent_cd, int dist, map<int,int>& freq) {
3     long long cnt = 0;
4     // This is called once per node; accumulate distances
5     for (int v : tree[u]) {
6         if (v == parent_cd || removed[v]) continue;
7         // recursively visit; actual implementation uses DFS collect + merge
8     }
9     return cnt;
10 }
11 // Full pattern: for each centroid c
12 // 1. collect dists[] of all reachable nodes (DFS, not crossing removed)
13 // 2. add dist=0 for the centroid itself
14 // 3. for each dist d in dists: answer += freq[K-d]; then freq[d]++
15 // 4. This counts paths through c in O(subtree size)
16 // Summed over all centroids: O(n log n) total

```

Complexity

Build: $\mathcal{O}(n \log n)$. All-paths query: $\mathcal{O}(n \log n)$ (with hashing) or $\mathcal{O}(n \log^2 n)$ (with sorted merging).

6.5 DSU on Tree (Small to Large / Dsu on Tree)

When to use

Why you need it:

You want subtree information that's **hard to express with simple DP**: e.g., count distinct colours in each subtree, most frequent element, etc.

Naive merge: at each node merge all children's sets $\mathcal{O}(n^2)$.

Small to Large (weighted union): Always merge the smaller set into the larger. Each element moves $\mathcal{O}(\log n)$ times total $\mathcal{O}(n \log^2 n)$ with ordered sets, $\mathcal{O}(n \log n)$ with hash maps.

Dsu on Tree ("Sack"): Reuse the heavy child's data structure. Light children are added then erased. $\mathcal{O}(n \log n)$.

```

1 // DSU on Tree (Sack) - O(n log n)
2 // Counts distinct values in each subtree
3 int col[100005]; // color of each node
4 map<int,int> *cur_map;
5 map<int,int> maps[100005];
6 int ans[100005];
7
8 void dfs_sack(int u, int p, bool keep) {
9     // Process light children first (their maps will be discarded)
10    for (int v : tree[u]) {
11        if (v == p || v == heavy[u]) continue;
12        dfs_sack(v, u, false); // don't keep light child's map
13    }
14    // Process heavy child, keep its map (inherit it)
15    if (heavy[u] != -1) {
16        dfs_sack(heavy[u], u, true);
17        cur_map = maps + heavy[u]; // inherit heavy child's map
18    } else {
19        cur_map = maps + u; // leaf: start fresh
20    }

```

```

21 // Add current node and all light children's subtrees to cur_map
22 (*cur_map)[col[u]]++;
23 for (int v : tree[u]) {
24     if (v == p || v == heavy[u]) continue;
25     for (auto [c, cnt] : maps[v]) (*cur_map)[c] += cnt;
26 }
27 ans[u] = cur_map->size(); // distinct colors in subtree of u
28 if (!keep) cur_map->clear(); // discard if light child
29 maps[u] = *cur_map;
30 }

```

Complexity

$\mathcal{O}(n \log n)$ time using the heavy-light structure (light subtrees are re-added at most $\mathcal{O}(\log n)$ times).

6.6 Rerooting DP

When to use

Why you need it:

Tree DP fixes a root and computes answers for all nodes *relative to that root* in $\mathcal{O}(n)$.

But what if you need the answer *for every possible root*? Re-running DP for each root = $\mathcal{O}(n^2)$.

Rerooting: Two-pass DFS. First pass computes `down[u]` (answer for subtree of u , rooted at 0). Second pass computes `up[u]` (contribution from the rest of the tree if u were root), propagated from parent.

Appears in: Sum of all distances, count paths through each node, tree DP with “virtual” rerooting.

```

1 // Classic example: sum of distances from each node to all others
2 // down[u] = sum of distances within subtree of u
3 // sz[u] = size of subtree of u
4 // ans[u] = sum of distances if u is root
5 int sz2[100005];
6 long long down2[100005], ans2[100005];
7
8 void dfs_down(int u, int p) {
9     sz2[u] = 1; down2[u] = 0;
10    for (int v : tree[u]) {
11        if (v == p) continue;
12        dfs_down(v, u);
13        sz2[u] += sz2[v];
14        down2[u] += down2[v] + sz2[v]; // each node in v's subtree is 1 step further
15    }
16 }
17
18 void dfs_up(int u, int p, int n) {
19     for (int v : tree[u]) {
20         if (v == p) continue;
21         // When we reroot at v:
22         // Nodes in v's subtree: sz2[v] of them, each 1 step closer
23         // Nodes outside v's subtree: n-sz2[v] of them, each 1 step further
24         ans2[v] = ans2[u] - sz2[v] + (n - sz2[v]);
25         dfs_up(v, u, n);
26     }
27 }
28 // Call: dfs_down(0, -1); ans2[0]=down2[0]; dfs_up(0, -1, n);
29 // ans2[u] = sum of distances from u to all other nodes

```

6.7 Virtual Tree (Auxiliary Tree)

When to use**Why you need it:**

You have q queries each specifying k important nodes and asking something about the minimal subtree connecting them.

Processing the whole n -node tree per query = $\mathcal{O}(qn)$. Too slow.

Virtual tree: Build a compressed tree of only the k key nodes + their pairwise LCAs $\mathcal{O}(k)$ nodes. Run DP on this small tree. Total: $\mathcal{O}((n + \sum k) \log n)$.

Appears in: Steiner tree problems, DP on a subset of tree nodes, competitive programming hard tree problems.

```

1  int tin_vt[100005]; // DFS in-time (Euler tour order)
2
3  // Build virtual tree from a set of key nodes
4  // Returns adjacency list of virtual tree
5  vector<pair<int,int>> buildVirtualTree(vector<int> keys, int root) {
6      sort(keys.begin(), keys.end(), [](int a, int b){ return tin_vt[a] < tin_vt[b]; });
7      int sz = keys.size();
8      // Add pairwise LCAs of consecutive nodes in DFS order
9      for (int i = 0; i + 1 < sz; i++) keys.push_back(lca(keys[i], keys[i+1]));
10     keys.push_back(root);
11     // Sort and deduplicate
12     sort(keys.begin(), keys.end(), [](int a, int b){ return tin_vt[a] < tin_vt[b]; });
13     keys.erase(unique(keys.begin(), keys.end()), keys.end());
14
15     // Reconstruct edges using a stack
16     vector<pair<int,int>> vtEdges;
17     stack<int> stk;
18     stk.push(keys[0]);
19     for (int i = 1; i < (int)keys.size(); i++) {
20         int v = keys[i];
21         int l = lca(v, stk.top());
22         if (l != stk.top()) {
23             while (stk.size() > 1 && tin_vt[stk.top()] > tin_vt[l]) {
24                 int child = stk.top(); stk.pop();
25                 vtEdges.push_back({stk.top(), child});
26             }
27             if (stk.top() != l) { vtEdges.push_back({l, stk.top()}); stk.pop(); stk.push(l); }
28         }
29         stk.push(v);
30     }
31     while (stk.size() > 1) { int c = stk.top(); stk.pop(); vtEdges.push_back({stk.top(), c}); }
32     return vtEdges;
33 }
```

6.8 Functional Graph

When to use**Why you need it:**

In a **functional graph** every node has exactly one outgoing edge ($f : V \rightarrow V$).

These appear naturally in: permutation cycles, “next element” problems, repeated function application.

Structure: each weakly connected component has exactly one cycle (the “rho” shape ρ).

Key operations: find cycle, find k -th step from v , detect cycle length.

```

1  // Floyd's tortoise and hare: find cycle length and entry point in  $\mathcal{O}(n)$ 
2  pair<int,int> floydCycle(int start, vector<int>& nxt) {
3      int slow = start, fast = start;
```

```

4   do { slow = nxt[slow]; fast = nxt[nxt[fast]]; } while (slow != fast);
5   // Cycle length
6   int cycleLen = 1; fast = nxt[slow];
7   while (fast != slow) { fast = nxt[fast]; cycleLen++; }
8   // Cycle entry point
9   slow = start; fast = start;
10  for (int i = 0; i < cycleLen; i++) fast = nxt[fast];
11  while (slow != fast) { slow = nxt[slow]; fast = nxt[fast]; }
12  return {slow, cycleLen}; // {entry point, cycle length}
13 }
14
15 // k-th step from v (using binary lifting on the functional graph)
16 int kthStep[100005][20]; // kthStep[v][k] = 2^k steps from v
17 void buildLifting(int n, vector<int>& nxt) {
18     for (int v = 0; v < n; v++) kthStep[v][0] = nxt[v];
19     for (int k = 1; k < 20; k++)
20         for (int v = 0; v < n; v++)
21             kthStep[v][k] = kthStep[kthStep[v][k-1]][k-1];
22 }
23 int jump(int v, long long steps) {
24     for (int k = 0; k < 20; k++) if ((steps >> k) & 1) v = kthStep[v][k];
25     return v;
26 }

```

6.9 Algorithm Selector Cheat Sheet

Problem	Algorithm	Complexity
Shortest path, unweighted	BFS	$O(V + E)$
Shortest path, weights $\{0, 1\}$	0-1 BFS	$O(V + E)$
Shortest path, non-neg weights	Dijkstra	$O(E \log V)$
Shortest path, neg weights	Bellman-Ford	$O(VE)$
Shortest path, DAG	Topo + DP	$O(V + E)$
All-pairs shortest path	Floyd-Warshall	$O(V^3)$
MST, sparse graph	Kruskal	$O(E \log E)$
MST, dense graph	Prim (array)	$O(V^2)$
Strongly connected components	Tarjan / Kosaraju	$O(V + E)$
Bridges / articulation points	Tarjan's DFS	$O(V + E)$
Max flow (general)	Dinic's	$O(V^2 E)$
Max flow (unit cap / bipartite)	Dinic's	$O(E\sqrt{V})$
Min-cost max-flow	SPFA + augment	$O(VEF)$
LCA query	Binary lifting	$O(\log n)$
Path query/update on tree	HLD + seg tree	$O(\log^2 n)$
Subtree query/update	Euler tour + BIT	$O(\log n)$
All-paths problem on tree	Centroid decomp.	$O(n \log n)$
Subtree aggregation (set/map)	DSU on Tree (Sack)	$O(n \log n)$
Answer for every root	Rerooting DP	$O(n)$
DP on a subset of tree nodes	Virtual tree	$O(k \log n)$
k -th step in functional graph	Binary lifting	$O(\log k)$
Euler circuit	Hierholzer's	$O(E)$

Chapter 7

Competitive Programming Graph Algorithms

7.1 2-SAT (Two-Satisfiability)

When to use

Why you need it:

You have n boolean variables and m clauses, each of the form $(x_i \vee x_j)$ (or negations). You need to decide if a satisfying assignment exists — and find it.

Key reduction: $(a \vee b)$ is equivalent to $(\neg a \Rightarrow b)$ AND $(\neg b \Rightarrow a)$. So each clause becomes two directed implications. Build an implication graph, find SCCs. If any variable x_i and $\neg x_i$ are in the *same* SCC: unsatisfiable.

Appears in: Scheduling, graph colouring feasibility, boolean constraint satisfaction.

```
1 // Variables: x_0 ... x_{n-1}
2 // Node 2*i = x_i is TRUE
3 // Node 2*i+1 = x_i is FALSE (negation)
4 struct TwoSAT {
5     int n;
6     vector<vector<int>> adj, radj;
7     vector<int> order, comp;
8     vector<bool> vis;
9
10    TwoSAT(int n) : n(n), adj(2*n), radj(2*n), comp(2*n), vis(2*n) {}
11
12    // Add clause (a OR b): a and b are literals (use lit(i,true/false))
13    int lit(int i, bool pos) { return 2*i + (pos ? 0 : 1); }
14    int neg(int x) { return x ^ 1; }
15
16    void addClause(int u, int v) { // u OR v
17        adj[neg(u)].push_back(v); // NOT u => v
18        adj[neg(v)].push_back(u); // NOT v => u
19        radj[v].push_back(neg(u));
20        radj[u].push_back(neg(v));
21    }
22
23    void dfs1(int u) { vis[u]=true; for(int v:adj[u]) if(!vis[v]) dfs1(v); order.push_back(u); }
24    void dfs2(int u, int c) { comp[u]=c; for(int v:radj[u]) if(comp[v]==-1) dfs2(v,c); }
25
26    // Returns true if satisfiable; fills 'val' with assignment
27    bool solve(vector<bool>& val) {
28        fill(vis.begin(),vis.end(),false);
29        for(int i=0;i<2*n;i++) if(!vis[i]) dfs1(i);
30        fill(comp.begin(),comp.end(),-1);
31        int c=0;
32        for(int i=2*n-1;i>=0;i--) if(comp[order[i]]==-1) dfs2(order[i],c++);
33        val.resize(n);
```

```

34     for(int i=0;i<n;i++){
35         if(comp[2*i]==comp[2*i+1]) return false; // contradiction
36         val[i] = comp[2*i] > comp[2*i+1]; // higher comp# = later in topo = true
37     }
38     return true;
39 }
40 };
41 // Usage:
42 // TwoSAT sat(n);
43 // sat.addClause(sat.lit(i, true), sat.lit(j, false)); // x_i OR NOT x_j
44 // vector<bool> val;
45 // if (sat.solve(val)) { /* val[i] is the assignment for x_i */ }

```

Complexity

$\mathcal{O}(V + E)$ via Kosaraju's SCC. Variables: n . Clauses: m graph has $2n$ nodes, $2m$ edges.

7.2 SCC Condensation & DAG DP

When to use

Why you need it:

After finding SCCs, contract each SCC into a single node. The result is a **DAG** (condensation graph). Any cyclic reachability problem becomes a DAG DP problem.

Use cases: Longest path in condensation, reachability queries, strongly connected reachability compression.

```

1 // After Tarjan/Kosaraju: sccId[v] = SCC id of vertex v
2 // Build condensation DAG, then run DP on it
3
4 vector<vector<int>> buildCondensation(int n, vector<vector<int>>& adj, vector<int>& sccId,
5 int numSCC) {
6     vector<vector<int>> dag(numSCC);
7     set<pair<int,int>> seen;
8     for (int u = 0; u < n; u++)
9         for (int v : adj[u])
10             if (sccId[u] != sccId[v] && !seen.count({sccId[u], sccId[v]})) {
11                 dag[sccId[u]].push_back(sccId[v]);
12                 seen.insert({sccId[u], sccId[v]});
13             }
14     return dag;
15 }
16
17 // Longest path in DAG (DP on topological order)
18 vector<int> longestPathDAG(int numSCC, vector<vector<int>>& dag) {
19     vector<int> indeg(numSCC, 0);
20     for (int u = 0; u < numSCC; u++) for (int v : dag[u]) indeg[v]++;
21     queue<int> q;
22     for (int i = 0; i < numSCC; i++) if (!indeg[i]) q.push(i);
23     vector<int> dp(numSCC, 0);
24     while (!q.empty()) {
25         int u = q.front(); q.pop();
26         for (int v : dag[u]) {
27             dp[v] = max(dp[v], dp[u] + 1);
28             if (--indeg[v] == 0) q.push(v);
29         }
30     }
31     return dp;
32 }

```


7.3 Dijkstra on State-Space Graphs

When to use

Why you need it:

Many CP problems have a **graph with extra state** beyond just the node: “shortest path using at most k edges”, “shortest path with fuel”, “shortest path in layered graph”.

Model the state as a node: $(node, state)$ is a new vertex in a state-space graph, then run Dijkstra.

Examples: shortest path with at most k edges (state = edges used), grid with k bombs available (state = bombs left), coin-state shortest path.

```

1 // Shortest path using at most K edges (layered graph)
2 // State: (node, edges_used) -- node in [0,n), edges in [0,K]
3 int dijkstraKEdges(int src, int dst, int n, int K, vector<vector<pair<int,int>>>& adj) {
4     // dist[node][k] = shortest distance to node using exactly k edges
5     vector<vector<int>> dist(n, vector<int>(K+1, INT_MAX));
6     using T = tuple<int,int,int>; // {dist, node, edges_used}
7     priority_queue<T, vector<T>, greater<T>> pq;
8     dist[src][0] = 0; pq.push({0, src, 0});
9     while (!pq.empty()) {
10         auto [d, u, k] = pq.top(); pq.pop();
11         if (d > dist[u][k]) continue;
12         if (k == K) continue; // can't use more edges
13         for (auto [v, w] : adj[u]) {
14             if (dist[u][k] + w < dist[v][k+1]) {
15                 dist[v][k+1] = dist[u][k] + w;
16                 pq.push({dist[v][k+1], v, k+1});
17             }
18         }
19     }
20     int ans = INT_MAX;
21     for (int k = 0; k <= K; k++) ans = min(ans, dist[dst][k]);
22     return ans;
23 }

```

```

1 // Shortest path with a choice: can teleport once for free
2 // State: (node, teleported?) -- 0 = not yet, 1 = already teleported
3 int dijkstraTeleport(int src, int dst, int n, vector<vector<pair<int,int>>>& adj,
4                     vector<vector<pair<int,int>>>& teleAdj) {
5     vector<array<int,2>> dist(n, {INT_MAX, INT_MAX});
6     using T = tuple<int,int,int>; // {dist, node, state}
7     priority_queue<T, vector<T>, greater<T>> pq;
8     dist[src][0] = 0; pq.push({0, src, 0});
9     while (!pq.empty()) {
10         auto [d, u, s] = pq.top(); pq.pop();
11         if (d > dist[u][s]) continue;
12         // Normal edge
13         for (auto [v,w] : adj[u]) if (dist[u][s]+w < dist[v][s]) {
14             dist[v][s] = dist[u][s]+w; pq.push({dist[v][s],v,s}); }
15         // Teleport (only if not yet used)
16         if (s == 0) for (auto [v,w] : teleAdj[u]) if (dist[u][0]+w < dist[v][1]) {
17             dist[v][1] = dist[u][0]+w; pq.push({dist[v][1],v,1}); }
18     }
19     return min(dist[dst][0], dist[dst][1]);
20 }

```

7.4 K-th Shortest Path (Yen's / Modified Dijkstra)

When to use**Why you need it:**

When the single shortest path isn't enough — some problems ask for the k -th shortest, or need fallback paths. Modified Dijkstra: allow each node to be *settled* up to k times.

```

1 // K-th shortest path from src to dst
2 // Allow each node to be popped from PQ up to K times
3 int kthShortest(int src, int dst, int K, int n, vector<vector<pair<int,int>>>& adj) {
4     vector<int> cnt(n, 0); // how many times node was settled
5     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
6     pq.push({0, src});
7     while (!pq.empty()) {
8         auto [d, u] = pq.top(); pq.pop();
9         cnt[u]++;
10        if (cnt[u] > K) continue; // settled K times: skip
11        if (u == dst && cnt[u] == K) return d; // K-th time we reach dst
12        for (auto [v, w] : adj[u])
13            if (cnt[v] < K) pq.push({d + w, v});
14    }
15    return -1; // fewer than K paths exist
16 }

```

Complexity

$\mathcal{O}(K \cdot E \log(KV))$. Works for any k but becomes slow for large k . Yen's algorithm gives K simple (loop-free) shortest paths in $\mathcal{O}(KV(V + E))$.

7.5 Hamiltonian Path (Bitmask DP)

When to use**Why you need it:**

A Hamiltonian path visits every vertex exactly once. It's NP-hard in general, but for small n (≤ 20) the bitmask DP solves it in $\mathcal{O}(2^n \cdot n^2)$.

State: $\text{dp}[\text{mask}][v]$ = can we visit exactly the vertices in mask , ending at v ?

```

1 // Returns true if Hamiltonian path exists; fills 'path'
2 bool hamiltonianPath(int n, vector<vector<bool>>& adj, vector<int>& path) {
3     // dp[mask][v] = true if we can visit vertices in mask, ending at v
4     vector<vector<bool>> dp(1<<n, vector<bool>(n, false));
5     vector<vector<int>> prev_v(1<<n, vector<int>(n, -1)); // for reconstruction
6
7     for (int v = 0; v < n; v++) dp[1<<v][v] = true;
8
9     for (int mask = 1; mask < (1<<n); mask++) {
10        for (int u = 0; u < n; u++) {
11            if (!dp[mask][u]) continue;
12            for (int v = 0; v < n; v++) {
13                if ((mask >> v) & 1) continue; // already visited
14                if (!adj[u][v]) continue; // no edge
15                int newMask = mask | (1<<v);
16                if (!dp[newMask][v]) {
17                    dp[newMask][v] = true;
18                    prev_v[newMask][v] = u;
19                }
20            }
21        }
22    }
23 }

```

```

23 int fullMask = (1<<n) - 1;
24 for (int v = 0; v < n; v++) {
25     if (!dp[fullMask][v]) continue;
26     // Reconstruct path
27     int cur = v, mask = fullMask;
28     while (cur != -1) {
29         path.push_back(cur);
30         int p = prev_v[mask][cur];
31         mask ^= (1<<cur);
32         cur = p;
33     }
34     reverse(path.begin(), path.end());
35     return true;
36 }
37 return false;
38 }

```

Complexity

$\mathcal{O}(2^n \cdot n^2)$ time, $\mathcal{O}(2^n \cdot n)$ space. Feasible for $n \leq 20$.

7.6 Tree DP Patterns

When to use

Why you need it:

Many tree problems require computing some aggregate over subtrees and combining at each node. These patterns appear constantly in contests — knowing them cold saves enormous time.

7.6.1 Tree Diameter

```

1 // Diameter = longest path between any two nodes (may not pass through root)
2 // For each node, track top-2 longest downward paths
3 int treeDiameter = 0;
4
5 int dfs_diam(int u, int p, vector<vector<pair<int,int>>>& adj) {
6     int longest = 0, second = 0;
7     for (auto [v, w] : adj[u]) {
8         if (v == p) continue;
9         int d = dfs_diam(v, u, adj) + w;
10        if (d >= longest) { second = longest; longest = d; }
11        else if (d > second) { second = d; }
12    }
13    treeDiameter = max(treeDiameter, longest + second); // path through u
14    return longest;
15 }

```

7.6.2 Max Independent Set on Tree

```

1 // dp[u][0] = max independent set in subtree of u, not including u
2 // dp[u][1] = max independent set in subtree of u, including u
3 int dp_mis[100005][2];
4
5 void dfs_mis(int u, int p) {
6     dp_mis[u][0] = 0; dp_mis[u][1] = 1; // 1 for including u itself
7     for (int v : tree[u]) {
8         if (v == p) continue;
9         dfs_mis(v, u);
10        dp_mis[u][0] += max(dp_mis[v][0], dp_mis[v][1]); // don't take u
11        dp_mis[u][1] += dp_mis[v][0]; // take u, can't take v

```

```

12     }
13 }
14 // Answer: max(dp_mis[root][0], dp_mis[root][1])

```

7.6.3 Counting Paths with Length Exactly K (Centroid DP)

```

1 // Using centroid decomposition:  $O(n \log^2 n)$ 
2 long long totalPairs = 0;
3 int K_target;
4
5 void collectDist(int u, int p, int d, vector<int>& dists) {
6     if (d > K_target) return;
7     dists.push_back(d);
8     for (int v : tree[u]) if (v != p && !removed[v]) collectDist(v, u, d+1, dists);
9 }
10
11 void solveCentroid(int u) {
12     calcSize(u, -1);
13     int c = getCentroid(u, -1, sz_cd[u]);
14     removed[c] = true;
15
16     map<int,int> freq; freq[0] = 1; // centroid itself at distance 0
17     for (int v : tree[c]) {
18         if (removed[v]) continue;
19         vector<int> dists;
20         collectDist(v, c, 1, dists);
21         // Count pairs: one endpoint in current subtree, one in previous subtrees
22         for (int d : dists) if (freq.count(K_target - d)) totalPairs += freq[K_target - d];
23         // Add current subtree's distances to freq
24         for (int d : dists) freq[d]++;
25     }
26     for (int v : tree[c]) if (!removed[v]) solveCentroid(v);
27 }

```

7.7 Graph Colouring

```

1 // Greedy colouring:  $O(V + E)$ , not always optimal
2 vector<int> greedyColor(int n, vector<vector<int>>& adj) {
3     vector<int> color(n, -1); color[0] = 0;
4     for (int u = 1; u < n; u++) {
5         set<int> used;
6         for (int v : adj[u]) if (color[v] != -1) used.insert(color[v]);
7         for (int c = 0; ; c++) if (!used.count(c)) { color[u] = c; break; }
8     }
9     return color;
10 }
11
12 // Chromatic number via bitmask DP (exact,  $O(2^n * n)$ )
13 // Independent sets: a set mask is independent if no two adjacent nodes in it
14 int chromaticNumber(int n, vector<vector<bool>>& adj) {
15     // is[mask] = true if mask is an independent set
16     vector<bool> isIndep(1<<n, true);
17     for (int mask = 1; mask < (1<<n); mask++) {
18         int u = __builtin_ctz(mask); // lowest set bit
19         for (int v = u+1; v < n; v++)
20             if ((mask>>v)&1 && adj[u][v]) { isIndep[mask]=false; break; }
21     }
22     // dp[mask] = min colours to colour vertices in mask
23     vector<int> dp(1<<n, n+1);
24     dp[0] = 0;
25     for (int mask = 1; mask < (1<<n); mask++) {
26         // Try all independent subsets of mask as one colour class
27         for (int sub = mask; sub; sub = (sub-1)&mask)
28             if (isIndep[sub]) dp[mask] = min(dp[mask], dp[mask^sub] + 1);

```

```

29     }
30     return dp[(1<n)-1];
31 }

```

Complexity

Greedy: $\mathcal{O}(V + E)$. Exact chromatic number: $\mathcal{O}(2^n \cdot n)$, feasible for $n \leq 20$.

7.8 Bipartite Matching Applications

7.8.1 Minimum Path Cover in DAG

Pattern / Core Idea

Minimum path cover = minimum number of vertex-disjoint paths that cover all nodes in a DAG.

Formula: Min path cover = $n - \text{Max bipartite matching}$.

Construction: Make bipartite graph: left node u , right node v , edge if DAG has $u \rightarrow v$.

```

1 // minPathCover = n - maxBipartiteMatching on the constructed bipartite graph
2 // Each matched edge (u->v) means u and v are in the same path
3 // Unmatched left nodes = path starts; unmatched right nodes = path ends
4 int minPathCover(int n, vector<pair<int,int>>& dagEdges) {
5     // Build bipartite graph: left[u] --- right[v] for each edge u->v in DAG
6     for (auto [u, v] : dagEdges) adjL[u].push_back(v);
7     return n - maxBipartiteMatching(n); // from ch5
8 }

```

7.8.2 Maximum Independent Set (Bipartite)

By König's theorem: Max Independent Set = $n - \text{Max Matching}$.

7.8.3 Minimum Vertex Cover (Bipartite)

By König's theorem: Min Vertex Cover = Max Matching.

7.9 Common Gotchas in Graph CP

Common Trap / Gotcha

1. Directed vs undirected: Always check — adding a reverse edge in a directed graph is wrong and vice versa. Many WAs come from this.

Common Trap / Gotcha

2. 0-indexed vs 1-indexed: Most contest graphs are 1-indexed. Initialize arrays of size $n + 1$, not n .

Common Trap / Gotcha

3. Disconnected graphs: BFS/DFS from a single source won't reach all nodes. Always loop over all unvisited nodes for components.

Common Trap / Gotcha

4. Integer overflow in Dijkstra: Use `long long` for distances when edge weights are large (e.g., $w \leq 10^9$, $n \leq 10^5$ $\max \text{dist} \approx 10^{14}$). Initialize `dist` with `1e18`, not `INT_MAX` (adding to `INT_MAX` overflows).

Common Trap / Gotcha

5. Self-loops in Floyd-Warshall: Initialize `dist[i][i] = 0`, not `INF`. A self-loop with negative weight makes `dist[i][i] < 0` — the negative cycle check.

Common Trap / Gotcha

6. Multiple edges (parallel edges): DSU doesn't care (Kruskal still works). But bridge-finding with parent tracking breaks — use edge-index tracking instead.

Common Trap / Gotcha

7. Flow graph re-use: After `maxflow()`, residual capacities are modified. If you need to run flow again, rebuild the graph.

Common Trap / Gotcha

8. LCA root: Always call `dfs_lca(root, root, 0)` (parent of root = root itself) so that `anc[root][k] = root` for all k — prevents out-of-bounds jumps during the lifting loop.